

Lab 5: Wireshark

Executive Summary

This report presents a network traffic analysis conducted using Wireshark and tshark on a vulnerable OWASP Metasploitable environment. The assessment focused on examining TCP/IP behavior, packet flags, and exposed network services. Key findings include successful TCP 3-way handshakes, exposure of open ports such as SSH (22), HTTP (80), and NetBIOS/SMB (139), and visibility of unencrypted HTTP traffic. The analysis highlights several security risks including exposed services and insecure communication protocols. Recommendations are provided to improve network security posture through encryption, service hardening, and network monitoring.

Objectives

- Capture and analyze network traffic using Wireshark/tshark.
- Identify different types of traffic (TCP, HTTP, HTTPS, etc.).
- Examine packet headers and TCP flags.
- Detect open services and potential vulnerabilities.
- Evaluate overall network security posture.

Methods Used

- **Tool used:** Wireshark / tshark
- Interface monitored: eth0
- Target system: OWASP Metasploitable (vulnerable Linux VM)

Capture Approach:

- Full packet capture (no initial filtering)
- Protocol-based filtering (TCP, HTTP analysis)

Focus Areas:

- TCP handshake behavior
- Service communication on exposed ports

- HTTP traffic inspection

Exercise 1:

Explore the Wireshark GUI. Identify and list the main components you see, including where to find the Statistics menu.

- **The Main component** in the GUI is the Menu Bar which contains: File, Edit, View, Capture, Analyze, Statistics, Telephony, Wireless, Tools, and Help.
- **Main Toolbar** Located below the Menu Bar contains: Quick-access icons for starting/stopping captures, opening files, saving captures, and applying filters.
- **Display Filter Bar** Located below the toolbar. Used to filter packets using expressions
- **The Statistics** menu can be found at the very top of the window. (Menu Bar)

Exercise 2:

- I Captured network traffic using both Wireshark and tshark. Compared the two methods and noted the differences in the user experience.

Screenshots of tshark

```

39 61.413302359 VMware_c0:00:08 → Broadcast ARP 60 Who has 192.168.5.2? Tell 192.168.5.1
40 62.413323188 VMware_c0:00:08 → Broadcast ARP 60 Who has 192.168.5.2? Tell 192.168.5.1
41 63.469583485 VMware_c0:00:08 → Broadcast ARP 60 Who has 192.168.5.2? Tell 192.168.5.1
42 64.414065641 VMware_c0:00:08 → Broadcast ARP 60 Who has 192.168.5.2? Tell 192.168.5.1
43 65.413733403 VMware_c0:00:08 → Broadcast ARP 60 Who has 192.168.5.2? Tell 192.168.5.1
44 66.476265948 VMware_c0:00:08 → Broadcast ARP 60 Who has 192.168.5.2? Tell 192.168.5.1
45 67.412724099 VMware_c0:00:08 → Broadcast ARP 60 Who has 192.168.5.2? Tell 192.168.5.1
46 68.412723880 VMware_c0:00:08 → Broadcast ARP 60 Who has 192.168.5.2? Tell 192.168.5.1
47 69.476060180 VMware_c0:00:08 → Broadcast ARP 60 Who has 192.168.5.2? Tell 192.168.5.1
48 70.413422830 VMware_c0:00:08 → Broadcast ARP 60 Who has 192.168.5.2? Tell 192.168.5.1
49 71.414938482 VMware_c0:00:08 → Broadcast ARP 60 Who has 192.168.5.2? Tell 192.168.5.1
50 80.576307485 VMware_4b:12:cf → Broadcast ARP 42 Who has 192.168.5.130? Tell 192.168.5.128
51 80.576996375 VMware_cf:2b:e9 → VMware_4b:12:cf ARP 60 192.168.5.130 is at 00:0c:29:cf:2b:e9
52 80.577005118 192.168.5.128 → 192.168.5.130 TCP 74 58784 → 80 [SYN] Seq=0 Win=64240 Len=0 MSS=1460 SACK_P
TSval=1519777314 TSect=0 WS=256
53 80.577599705 192.168.5.130 → 192.168.5.128 TCP 74 80 → 58784 [SYN, ACK] Seq=0 Ack=1 Win=5792 Len=0 MSS=1
SACK_PERM TSval=6836359 TSect=1519777314 WS=64
54 80.577916139 192.168.5.128 → 192.168.5.130 TCP 66 58784 → 80 [ACK] Seq=1 Ack=1 Win=64256 Len=0 TSval=151
7316 TSect=6836359
55 80.578252040 192.168.5.128 → 192.168.5.130 HTTP 790 POST /mutillidae/index.php?page=login.php HTTP/1.1
plication/x-www-form-urlencoded)
56 80.579109845 192.168.5.130 → 192.168.5.128 TCP 66 80 → 58784 [ACK] Seq=1 Ack=725 Win=7296 Len=0 TSval=68
60 TSect=1519777316
57 80.595971523 192.168.5.130 → 192.168.5.128 TCP 1514 HTTP/1.1 200 OK
58 80.596082026 192.168.5.128 → 192.168.5.130 TCP 66 58784 → 80 [ACK] Seq=725 Ack=1449 Win=67328 Len=0 TSva
519777334 TSect=6836364
59 80.596283011 192.168.5.130 → 192.168.5.128 TCP 1514 80 → 58784 [ACK] Seq=1449 Ack=725 Win=7296 Len=1448
al=6836364 TSect=1519777316
60 80.596290863 192.168.5.128 → 192.168.5.130 TCP 66 58784 → 80 [ACK] Seq=725 Ack=2897 Win=70144 Len=0 TSva
519777334 TSect=6836364
61 80.596730028 192.168.5.130 → 192.168.5.128 TCP 1514 80 → 58784 [ACK] Seq=2897 Ack=725 Win=7296 Len=1448
al=6836364 TSect=1519777316
62 80.596734514 192.168.5.128 → 192.168.5.130 TCP 66 58784 → 80 [ACK] Seq=725 Ack=4345 Win=72960 Len=0 TSva
519777334 TSect=6836364
63 80.597371238 192.168.5.130 → 192.168.5.128 TCP 1514 80 → 58784 [ACK] Seq=4345 Ack=725 Win=7296 Len=1448

```

and wireshark

The image shows a Wireshark network traffic capture. The top pane displays a list of captured packets, with the DNS section expanded. The middle pane shows the details of the selected packet (Frame 33), which is a Domain Name System (query) packet. The bottom pane shows the raw packet data in hexadecimal and ASCII.

No.	Source	Destination	Protocol	Length	Info
36	192.168.5.128	192.168.5.2	DNS	97	Standard query 0xe58a A firefox.settings.services.mozilla.com
37	192.168.5.128	192.168.5.2	DNS	109	Standard query 0x984e A firefox.settings.services.mozilla.com.localdomain
42	192.168.5.2	192.168.5.128	DNS	149	Standard query response 0xe58a A firefox.settings.services.mozilla.com CNAME mozilla.map.fastly.net A 146.75...
43	192.168.5.128	192.168.5.2	ICMP	177	Destination unreachable (Port unreachable)
44	192.168.5.2	192.168.5.128	DNS	161	Standard query response 0x984e A firefox.settings.services.mozilla.com.localdomain CNAME mozilla.map.fastly...
45	192.168.5.128	192.168.5.2	DNS	109	Standard query 0x5c70 AAAA firefox.settings.services.mozilla.com.localdomain
46	192.168.5.128	192.168.5.2	DNS	87	Standard query 0x4a75 A safebrowsing.googleapis.com
50	192.168.5.128	192.168.5.2	DNS	109	Standard query 0x984e A firefox.settings.services.mozilla.com.localdomain
51	192.168.5.2	192.168.5.128	DNS	161	Standard query response 0x984e A firefox.settings.services.mozilla.com.localdomain CNAME mozilla.map.fastly...
52	192.168.5.128	192.168.5.2	DNS	109	Standard query 0x5c70 AAAA firefox.settings.services.mozilla.com.localdomain
55	192.168.5.128	192.168.5.2	DNS	76	Standard query 0xb374 HTTPS learn.uchent.com
56	192.168.5.128	192.168.5.2	DNS	76	Standard query 0x699e A learn.uchent.com
57	192.168.5.128	192.168.5.2	DNS	76	Standard query 0x659d AAAA learn.uchent.com
58	192.168.5.128	192.168.5.2	DNS	74	Standard query 0x7a67 A www.google.com
59	192.168.5.128	192.168.5.2	DNS	74	Standard query 0xde65 AAAA www.google.com
60	192.168.5.2	192.168.5.128	DNS	282	Standard query response 0x7a67 A www.google.com A 142.251.157.119 A 142.251.155.119 A 142.251.156.119 A 142...
61	192.168.5.128	192.168.5.2	DNS	74	Standard query 0xe292 A www.google.com

Frame 33: Packet, 97 bytes on wire (776 bits), 97 bytes captured (776 bits) on interface eth0
Ethernet II, Src: VMware_4b:12:cf (00:0c:29:4b:12:cf), Dst: VMware_e2:35:4d (00:50:56:e2:35:4d)
Internet Protocol Version 4, Src: 192.168.5.128, Dst: 192.168.5.2
User Datagram Protocol, Src Port: 46831, Dst Port: 53
Domain Name System (query)

tshark (Terminal)

- Command-line version of Wireshark
- Shows raw text output of packets
- Faster and lighter (no GUI)
- Requires commands and filters to analyze

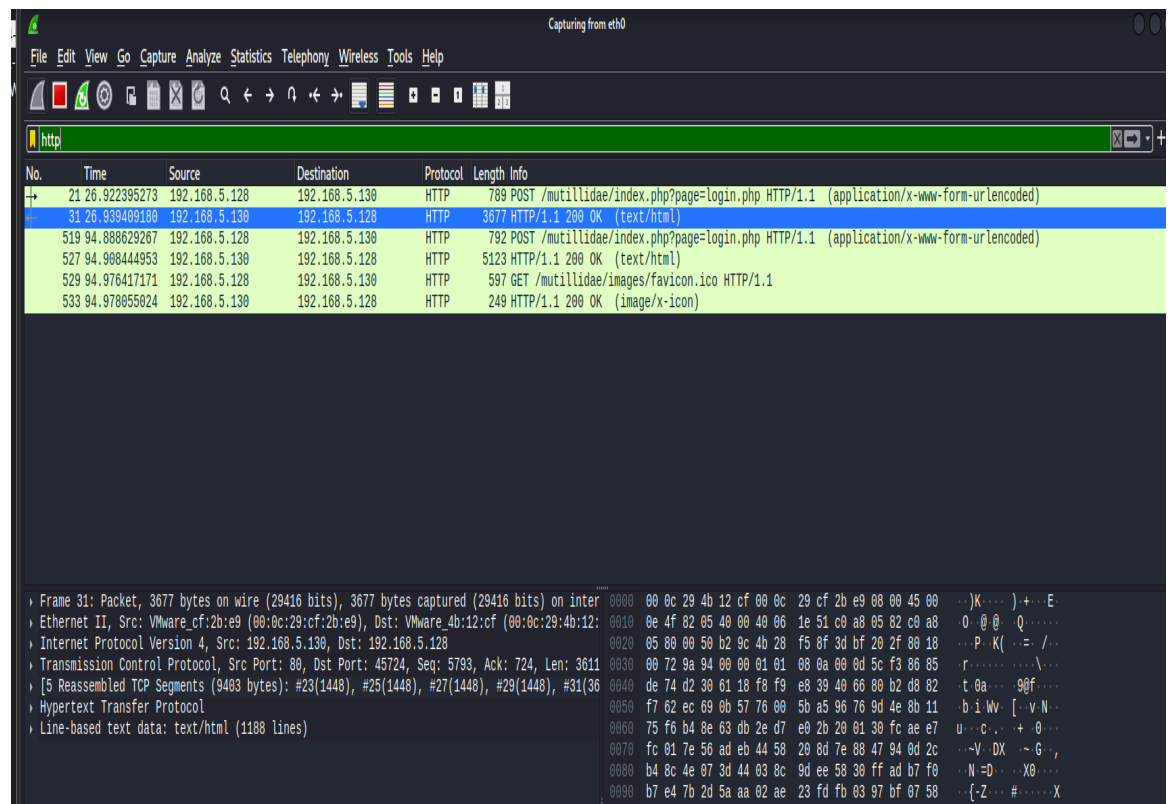
Wireshark (GUI)

- Visual interface with colors and graphs
- Easy to click packets and inspect details
- Shows packet flow in layers (Ethernet → IP → TCP → HTTP)
- You can filter using buttons and display filters,

Example:one can click the HTTP packet.

Exercise 3:

- Use filters to analyze different types of traffic. Record the following:
 - The Number of HTTP packets captured: 6 packets



.- Number of DNS packets captured: 69 packets

No.	Time	Source	Destination	Protocol	Length	Info
36	31.986669990	192.168.5.128	192.168.5.2	DNS	97	Standard query 0xe50a A firefox.settings.services.mozilla.com
37	36.989418997	192.168.5.128	192.168.5.2	DNS	109	Standard query 0x984e A firefox.settings.services.mozilla.com.localdomain
42	38.015788642	192.168.5.2	192.168.5.128	DNS	149	Standard query response 0xe50a A firefox.settings.services.mozilla.com CNAME mozilla.map.fastly.net A 146.75...
43	38.015849710	192.168.5.128	192.168.5.2	ICMP	177	Destination unreachable (Port unreachable)
44	38.288334690	192.168.5.2	192.168.5.128	DNS	161	Standard query response 0x984e A firefox.settings.services.mozilla.com.localdomain CNAME mozilla.map.fastly...
45	38.282265822	192.168.5.128	192.168.5.2	DNS	109	Standard query 0x5c70 AAAA firefox.settings.services.mozilla.com.localdomain
46	41.665228563	192.168.5.128	192.168.5.2	DNS	87	Standard query 0x4a75 A safebrowsing.googleapis.com
50	41.992588589	192.168.5.128	192.168.5.2	DNS	109	Standard query 0x984e A firefox.settings.services.mozilla.com.localdomain
51	41.993912473	192.168.5.2	192.168.5.128	DNS	161	Standard query response 0x984e A firefox.settings.services.mozilla.com.localdomain CNAME mozilla.map.fastly...
52	41.993984086	192.168.5.128	192.168.5.2	DNS	109	Standard query 0x5c70 AAAA firefox.settings.services.mozilla.com.localdomain
55	45.769623980	192.168.5.128	192.168.5.2	DNS	76	Standard query 0xb374 HTTPS learn.uchent.com
56	45.769718425	192.168.5.128	192.168.5.2	DNS	76	Standard query 0x699e A learn.uchent.com
57	45.769719318	192.168.5.128	192.168.5.2	DNS	76	Standard query 0x659d AAAA learn.uchent.com
58	46.115575896	192.168.5.128	192.168.5.2	DNS	74	Standard query 0x7a67 A www.google.com
59	46.115663855	192.168.5.128	192.168.5.2	DNS	74	Standard query 0xde65 AAAA www.google.com
60	46.120704432	192.168.5.2	192.168.5.128	DNS	282	Standard query response 0x7a67 A www.google.com A 142.251.157.119 A 142.251.155.119 A 142.251.156.119 A 142...
61	46.226871160	192.168.5.128	192.168.5.2	DNS	74	Standard query 0xe292 A www.google.com

Frame 33: Packet, 97 bytes on wire (776 bits), 97 bytes captured (776 bits) on interface eth0
Ethernet II, Src: VMware_4b:12:cf (00:0c:29:4b:12:cf), Dst: VMware_e2:35:4d (00:50:56:e2:35:4d)
Internet Protocol Version 4, Src: 192.168.5.128, Dst: 192.168.5.2
User Datagram Protocol, Src Port: 46831, Dst Port: 53
Domain Name System (query)

Specific IP addresses you

identified in the traffic:

- 192.168.5.128 (eth0 ip) and 192.168.5.130 (owasp ip)
- 34.107.243.93,34.107.221.82 (External/Public IPs)

Exercise 4:

- Select a packet and list the following information:

No.	Time	Source	Destination	Protocol	Length	Info
21	26.922395273	192.168.5.128	192.168.5.130	HTTP	789	POST /mutillidae/index.php

- Source IP:192.168.5.128
- Destination IP: 192.168.5.130
- Protocol: HTTP
- Any TCP Flags observed: ACK(Acknowledge receipt of packets),FIN

and ACK

Screenshot of TCP observerd.

No.	Time	Source	Destination	Protocol	Length	Info
5531	6896.7118405...	34.107.243.93	192.168.5.128	TLSv1.3	191	Application Data
5532	6896.7603956...	192.168.5.128	34.107.243.93	TCP	54	45810 → 443 [ACK] Seq=2070 Ack=1163 Win=63078 Len=0
5552	6944.5895634...	192.168.5.128	142.251.157.119	TLSv1.3	93	Application Data
5553	6944.5902746...	142.251.157.119	192.168.5.128	TCP	60	443 → 40302 [ACK] Seq=1991 Ack=2762 Win=64240 Len=0
5554	6952.7267801...	192.168.5.128	142.251.157.119	TLSv1.3	93	Application Data
5555	6952.7271988...	142.251.157.119	192.168.5.128	TCP	60	443 → 40302 [ACK] Seq=1991 Ack=2801 Win=64240 Len=0
5556	6952.7274135...	192.168.5.128	142.251.157.119	TLSv1.3	78	Application Data
5557	6952.7275450...	192.168.5.128	142.251.157.119	TCP	54	40302 → 443 [FIN, ACK] Seq=2825 Ack=1991 Win=62250 Len=0
5558	6952.7278351...	142.251.157.119	192.168.5.128	TCP	60	443 → 40302 [ACK] Seq=1991 Ack=2825 Win=64240 Len=0
5559	6952.7278864...	142.251.157.119	192.168.5.128	TCP	60	443 → 40302 [ACK] Seq=1991 Ack=2826 Win=64239 Len=0
5560	6953.7296496...	192.168.5.128	34.107.243.93	TLSv1.3	93	Application Data
5561	6953.7300407...	34.107.243.93	192.168.5.128	TCP	60	443 → 45804 [ACK] Seq=862 Ack=2752 Win=64240 Len=0
5562	6961.7697221...	192.168.5.128	34.107.243.93	TLSv1.3	93	Application Data
5563	6961.7701593...	34.107.243.93	192.168.5.128	TCP	60	443 → 45804 [ACK] Seq=862 Ack=2791 Win=64240 Len=0
5564	6961.7706317...	192.168.5.128	34.107.243.93	TLSv1.3	78	Application Data
5565	6961.7708350...	192.168.5.128	34.107.243.93	TCP	54	45804 → 443 [FIN, ACK] Seq=2815 Ack=862 Win=63379 Len=0
5566	6961.7709510...	34.107.243.93	192.168.5.128	TCP	60	443 → 45804 [ACK] Seq=862 Ack=2815 Win=64240 Len=0
5567	6961.7721550...	34.107.243.93	192.168.5.128	TCP	60	443 → 45804 [ACK] Seq=862 Ack=2816 Win=64239 Len=0
5570	6985.2704599...	142.251.157.119	192.168.5.128	TCP	60	443 → 40302 [RST, ACK] Seq=1991 Ack=2826 Win=64239 Len=0
5571	6988.0613536...	34.107.243.93	192.168.5.128	TCP	60	443 → 45804 [RST, ACK] Seq=862 Ack=2816 Win=64239 Len=0
5676	7196.7952104...	34.107.243.93	192.168.5.128	TLSv1.3	78	Application Data
5677	7196.7952412...	192.168.5.128	34.107.243.93	TCP	54	45810 → 443 [ACK] Seq=2070 Ack=1187 Win=63054 Len=0
5678	7196.7954469...	192.168.5.128	34.107.243.93	TLSv1.3	82	Application Data
5679	7196.7958428...	34.107.243.93	192.168.5.128	TCP	60	443 → 45810 [ACK] Seq=1187 Ack=2098 Win=64240 Len=0

Exercise 5:

- Follow a TCP stream for a specific session and summarize the data exchanged between the client and server.

Wireshark - Packet 2662: eth0

tcp.stream eq 49

No. Time Source Destination Protocol Length Info

2417 3203.9652436... 192.168.5.128 34.107.243.93 TLSv1.3 191 Application Data

2418 3203.9653367... 192.168.5.128 34.107.243.93 TCP 54 45810 → 443 [ACK] Seq=2070 Ack=1163 Win=63078 Len=0

2421 3205.1729226... 192.168.5.128 142.251.157.119 TLSv1.3 93 Application Data

2422 3205.1731425... 192.168.5.128 142.251.157.119 TLSv1.3 93 Application Data

2428 3207.5774314... 192.168.5.128 142.251.157.119 TCP 54 40302 → 443 [FIN, ACK] Seq=2825 Ack=1991 Win=62250 Len=0

2429 3207.5778201... 142.251.157.119 192.168.5.128 TCP 60 443 → 40302 [ACK] Seq=1991 Ack=2825 Win=64240 Len=0

2626 3222.3307503... 192.168.5.128 142.251.157.119 TCP 60 443 → 40302 [ACK] Seq=1991 Ack=2826 Win=64240 Len=0

2637 3222.8825553... 142.251.157.119 192.168.5.128 TCP 60 443 → 40302 [ACK] Seq=1991 Ack=2826 Win=64240 Len=0

2638 3222.8825553... 142.251.157.119 192.168.5.128 TCP 60 443 → 40302 [ACK] Seq=1991 Ack=2826 Win=64240 Len=0

2642 3222.8833065... 192.168.5.128 142.251.157.119 TCP 60 443 → 40302 [ACK] Seq=1991 Ack=2826 Win=64240 Len=0

2656 3222.9916152... 142.251.157.119 192.168.5.128 TCP 60 443 → 40302 [ACK] Seq=1991 Ack=2826 Win=64240 Len=0

2657 3222.9938183... 192.168.5.128 142.251.157.119 TCP 60 443 → 40302 [ACK] Seq=1991 Ack=2826 Win=64240 Len=0

2662 3223.0323349... 142.251.157.119 192.168.5.128 TCP 60 443 → 40302 [RST, ACK] Seq=1991 Ack=2826 Win=64239 Len=0

2663 3223.0330275... 192.168.5.128 142.251.157.119 TCP 60 443 → 40302 [RST, ACK] Seq=1991 Ack=2826 Win=64239 Len=0

2667 3223.0481504... 142.251.157.119 192.168.5.128 TCP 60 443 → 40302 [RST, ACK] Seq=1991 Ack=2826 Win=64239 Len=0

2668 3223.0485914... 192.168.5.128 142.251.157.119 TCP 60 443 → 40302 [RST, ACK] Seq=1991 Ack=2826 Win=64239 Len=0

2675 3223.0817078... 142.251.157.119 192.168.5.128 TCP 60 443 → 40302 [RST, ACK] Seq=1991 Ack=2826 Win=64239 Len=0

2676 3223.0820352... 192.168.5.128 142.251.157.119 TCP 60 443 → 40302 [RST, ACK] Seq=1991 Ack=2826 Win=64239 Len=0

2677 3223.0885008... 142.251.157.119 192.168.5.128 TCP 60 443 → 40302 [RST, ACK] Seq=1991 Ack=2826 Win=64239 Len=0

2678 3223.0915320... 192.168.5.128 142.251.157.119 TCP 60 443 → 40302 [RST, ACK] Seq=1991 Ack=2826 Win=64239 Len=0

2679 3223.0996908... 192.168.5.128 142.251.157.119 TCP 60 443 → 40302 [RST, ACK] Seq=1991 Ack=2826 Win=64239 Len=0

2680 3223.0998549... 192.168.5.128 142.251.157.119 TCP 60 443 → 40302 [RST, ACK] Seq=1991 Ack=2826 Win=64239 Len=0

2681 3223.1001532... 192.168.5.128 142.251.157.119 TCP 60 443 → 40302 [RST, ACK] Seq=1991 Ack=2826 Win=64239 Len=0

2691 3223.1525849... 142.251.157.119 192.168.5.128 TCP 60 443 → 40302 [RST, ACK] Seq=1991 Ack=2826 Win=64239 Len=0

2693 3223.1533198... 192.168.5.128 142.251.157.119 TCP 60 443 → 40302 [RST, ACK] Seq=1991 Ack=2826 Win=64239 Len=0

Frame 2662: Packet, 1294 bytes on wire (10352 bits), 1294 bytes captured (10352 bits) on interface eth0, id 0

Section number: 1

Interface id: 0 (eth0)

Encapsulation type: Ethernet (1)

Arrival Time: May 23, 2026 11:43:33.465097753 EDT

UTC Arrival Time: May 23, 2026 15:43:33.465097753 UTC

Epoch Arrival Time: 1779551013.465097753

[Time shift for this packet: 0.000000000 seconds]

[Time delta from previous captured frame: -163.296 microseconds]

[Time delta from previous displayed frame: 38.516561 milliseconds]

[Time since reference or first frame: 53 minutes, 43.032334917 seconds]

Frame Number: 2662

Frame Length: 1294 bytes (10352 bits)

Capture Length: 1294 bytes (10352 bits)

[Frame is marked: False]

0000 00 0c 29 4b 12 cf 00 50 56 e2 35 4d 08 00 45 00 ...K...P V 5M...E

0010 05 00 49 8f 00 00 80 11 fe c2 8e fb 98 77 c0 a8 ...I...w...

0020 05 00 01 bb d3 2a 04 ec 49 84 cc 00 00 00 01 03 ...*...I...

0030 36 97 5c 08 ee 6a ea a0 d7 98 21 f5 00 40 20 0e 6 \ j . ! . @

0040 e8 45 87 e5 5c 4f c5 bb 05 63 ac 9d 65 5f fc f1 E \0 . c . e

0050 de 11 01 b6 11 80 bf 98 64 e6 e4 a3 06 46 1e e4 . . . d . . F

0060 00 00 00 01 03 36 97 5c 08 ee 6a ea a0 d7 98 21 . . . 6 \ j . !

0070 f5 44 9b 51 0c b6 2e a4 ab 08 e6 2f e0 70 d6 63 . D . . . / . p . c

0080 08 0e 52 c8 0f f6 04 77 70 77 d4 fd 25 67 99 be . R . . . w pw . %W .

0090 eb 2d c8 61 3d 6f b8 9a 7e d1 ac 60 85 cf 3c 49 . . a < I

00a0 83 c3 e1 ad e8 7a d6 c1 f8 7e ed fb 19 d0 f7 9f . . . z

00b0 e8 2e 97 d6 95 45 28 48 2c 4f 7e 32 21 62 10 17 . . . E (H , 0 - 2 ! b .

00c0 59 50 65 52 bf 35 66 7c a6 14 c2 09 1f ab 4c 5a Y P e r . 5 f | L Z

1. The analyzed TCP stream (tcp.stream eq 49) showed active communication between the local host 192.168.5.128 and an external/public internet server 142.125.152.119 using Internet Protocol Version 4 (IPv4).
2. The capture used Ethernet II encapsulation on the eth0 network interface. The encapsulation type was identified as Ethernet (1).
3. The stream contained TCP packets with relatively large payload sizes of up to 1294 bytes. The frame length was also 1294 bytes, indicating the transfer of application data rather than simple acknowledgment packets(ACK).
4. The packet contents appeared unreadable and randomized, suggesting that the communication was encrypted using TLS/HTTPS protocols. The Time To Live (TTL) value observed was 128.
5. Timing analysis showed a fast response interval of approximately 38 ms between packets, indicating stable and responsive network Communication.

Exercise 6:

- Take a screenshot of the Protocol Hierarchy and analyze the data.

Wireshark - Protocol Hierarchy Statistics - eth0

Protocol	Percent Packets	Packets	Percent Bytes	Bytes	Bits/s	End Packets	End Bytes	End Bits/s	PDUs
Frame	100.0	9338	100.0	1641818	1,153	0	0	0	9338
Ethernet	100.0	9338	11.4	187056	131	0	0	0	9338
Internet Protocol Version 6	9.8	911	2.4	38736	27	0	0	0	911
User Datagram Protocol	6.2	575	0.3	4600	3	0	0	0	575
Multicast Domain Name System	4.2	390	0.9	15568	10	390	15568	10	390
Link-local Multicast Name Resolution	2.0	185	0.4	6308	4	185	6308	4	185
Internet Control Message Protocol v6	3.6	336	0.5	8648	6	336	8648	6	336
Internet Protocol Version 4	58.0	5417	6.7	109360	76	0	0	0	5417
User Datagram Protocol	25.3	2360	1.1	18880	13	0	0	0	2360
Simple Service Discovery Protocol	1.1	107	0.9	14659	10	107	14659	10	107
QUIC IETF	4.5	423	17.1	280203	196	422	231521	162	546
Malformed Packet	0.0	1	0.0	0	0	1	0	0	1
Network Time Protocol	6.3	586	1.7	28128	19	586	28128	19	586
NetBIOS Name Service	0.1	12	0.0	600	0	12	600	0	12
NetBIOS Datagram Service	0.3	32	0.2	2624	1	0	0	0	32
SMB (Server Message Block Protocol)	0.3	32	0.3	4432	3	0	0	0	32
SMB MailSlot Protocol	0.3	32	0.0	800	0	0	0	0	32
Microsoft Windows Browser Protocol	0.3	32	0.1	1680	1	32	1680	1	32
Multicast Domain Name System	4.2	390	0.9	15568	10	390	15568	10	390
Link-local Multicast Name Resolution	2.0	185	0.4	6308	4	185	6308	4	185
Dynamic Host Configuration Protocol	1.0	94	1.7	28152	19	94	28152	19	94
Domain Name System	5.7	531	2.3	37000	25	531	37000	25	531
Transmission Control Protocol	28.8	2693	3.6	58664	41	1764	40012	28	2693
Transport Layer Security	9.7	907	40.8	669390	470	907	629582	442	915
Hypertext Transfer Protocol	0.2	22	0.5	8594	6	9	3011	2	22
Media Type	0.0	1	0.1	1150	0	1	1150	0	1
Line-based text data	0.1	10	6.1	100732	70	10	100732	70	10
HTML Form URL Encoded	0.0	2	0.0	121	0	2	121	0	2
Internet Group Management Protocol	2.7	255	0.3	4120	2	255	4120	2	255
Internet Control Message Protocol	1.2	109	0.6	10161	7	109	10161	7	109

No display filter.

Close Copy Protocols Help

- Which protocol is most prevalent in your capture?

The Protocol Hierarchy Statistics showed that the Transmission Control Protocol (TCP) was the most prevalent protocol in network capture, accounting for 28.8% of all packets (2693 packets).

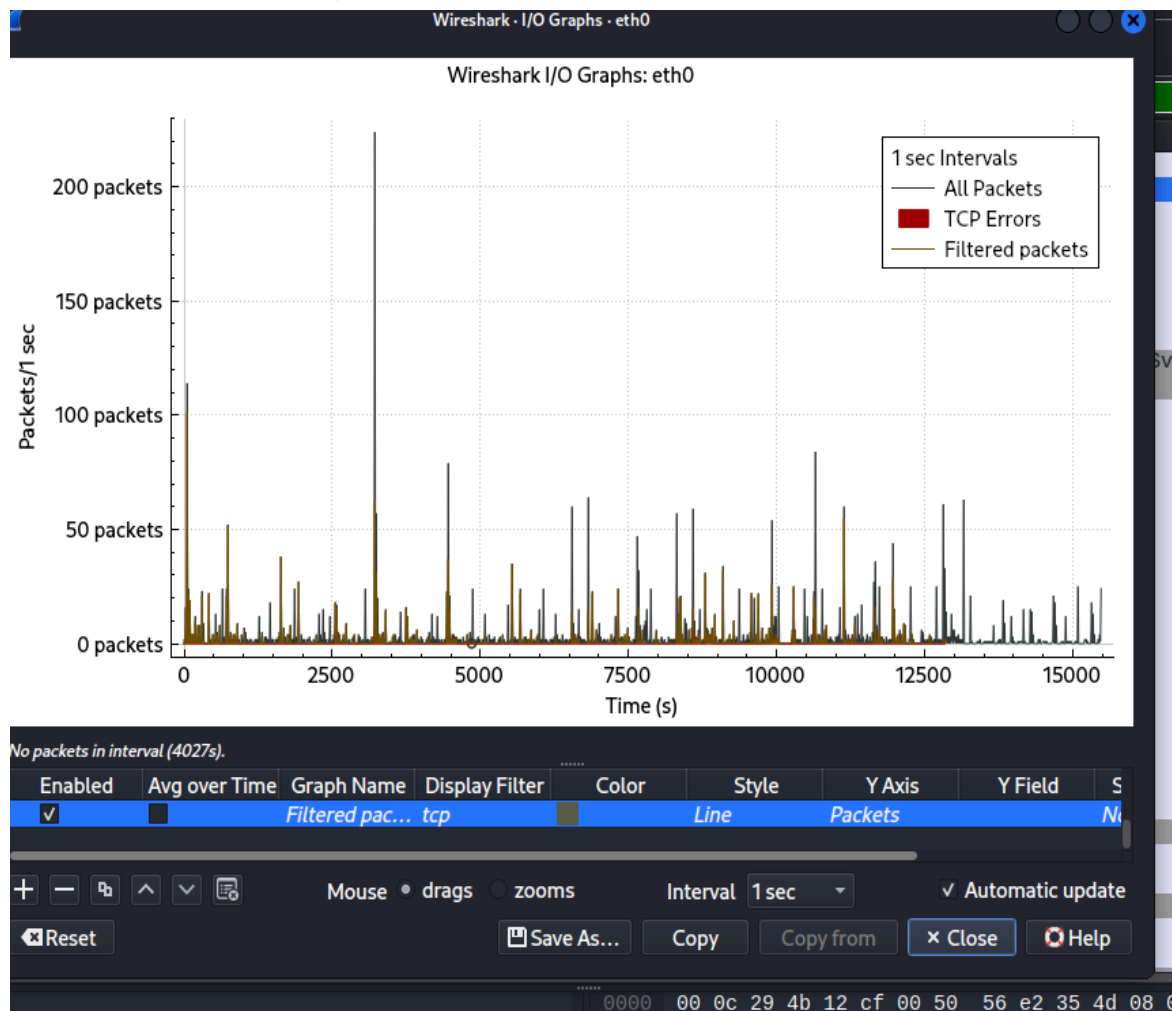
- IO Graphs:

- Access Statistics > IO Graphs to visualize traffic over time.

Exercise 7:

- Create an IO Graph showing TCP traffic. Describe any noticeable patterns you observe:

Screenshot of IO Graphs:



Noticeable Patterns Observed:

The IO Graph showed a smooth and continuous flow of network traffic over time, indicating stable and normal communication between hosts.

Several sudden sharp upward peaks were also observed in the graph. These traffic spikes suggest periods of increased network activity and high packet transmission, likely caused by encrypted TLS/HTTPS communication, DNS query during internet access

.Exercise 8:

- Save your capture file and describe a scenario where you would need to review this data later. What specific findings do you hope to extract?

A situation where this data may need to be reviewed later is during the investigation of suspicious network behavior, slow internet performance, or unauthorized communication with external servers.

From the captured traffic, I would hope to extract:

- Source and destination IP addresses involved in communication,
- TCP sessions and connection details,
- HTTP and TLS/HTTPS activity,
- DNS requests and responses,
- Packet timing and response delays,
- Indicators of suspicious or malicious traffic.

Exercise 9:

- Describe a real-world scenario where you would use Wireshark to troubleshoot a network issue. What specific symptoms would you Investigate?

- A real-world scenario for using Wireshark would be troubleshooting slow or unstable internet connectivity in an organization or home network. For example, users may complain that websites load slowly

or online applications frequently disconnect.

Using Wireshark, the investigation would focus on symptoms such as:

- High packet loss,
- Excessive TCP retransmissions,
- DNS resolution failures,
- Delayed server responses,
- Connection resets (RST packets),
- High latency between client and server,
- Unusual traffic spikes or bandwidth usage.

Wireshark would help identify whether the issue is caused by the local device, router, ISP, or external servers.

Exercise 10:

Identify at least two potential security threats in your captured traffic.
What indicators led you to suspect these activities?

Two potential security threats observed in the captured traffic include:

1. Unencrypted HTTP Traffic

The capture contained HTTP packets such as:

- GET /success.txt?ipv4 HTTP/1.1

This is a potential security risk because HTTP traffic is transmitted in plain text and may expose sensitive information to attackers performing packet sniffing or man-in-the-middle attacks.

2. Communication with External Public Servers

The system communicated with external public IP addresses such as:

- 34.107.243.93
- 34.107.221.82
- 142.125.152.119

Frequent encrypted communication with unknown external servers may indicate:

- background application activity.
- cloud service connections
- or potentially suspicious outbound traffic.

The indicators that raised attention included:

- multiple TLS sessions,
- repeated TCP handshakes.

- continuous data exchange with external hosts,
- and high numbers of ACK and PSH/ACK packets.

Although no direct malicious activity was confirmed, these patterns would require further investigation in a real security analysis.

Lab 6: Advanced Packet Analysis Techniques

Exercise 1:

- Describe the purpose of the SYN and ACK flags in the TCP handshake. How do these flags indicate the status of a connection?

The image shows a Wireshark packet capture analysis of a TCP connection. The packet list on the left shows several packets, with the selected packet (No. 517) highlighted in blue. The packet details pane on the right shows the structure of the selected packet, including the Ethernet II header, Internet Protocol Version 4 header, and Transmission Control Protocol header. The packet bytes pane at the bottom shows the raw data of the selected packet.

No.	Time	Source	Destination	Protocol	Length	Info
517	94.888364928	192.168.5.130	192.168.5.128	TCP	74	80 → 40158 [SYN, ACK] Seq=0 Ack=1 Win=5792 Len=0 MSS=1460 SACK_PERM TSval=892791 TSecr=3119287179 WS=64
518	94.888444496	192.168.5.128	192.168.5.130	TCP	66	40158 → 80 [ACK] Seq=1 Ack=1 Win=64256 Len=0 TSval=3119287179 TSecr=892791
519	94.888629267	192.168.5.128	192.168.5.130	HTTP	792	POST /mutillidae/index.php?page=login.php HTTP/1.1 (application/x-www-form-urlencoded)
520	94.889632813	192.168.5.130	192.168.5.128	TCP	66	80 → 40158 [ACK] Seq=1 Ack=727 Win=7296 Len=0 TSval=892791 TSecr=3119287179
521	94.907646652	192.168.5.130	192.168.5.128	TCP	1514	80 → 40158 [ACK] Seq=1 Ack=727 Win=7296 Len=1448 TSval=892795 TSecr=3119287179 [TCP PDU reassembled in 52]
522	94.907681564	192.168.5.128	192.168.5.130	TCP	66	40158 → 80 [ACK] Seq=727 Ack=1449 Win=67328 Len=0 TSval=3119287198 TSecr=892795
523	94.907167912	192.168.5.130	192.168.5.128	TCP	1514	80 → 40158 [ACK] Seq=1449 Ack=727 Win=7296 Len=1448 TSval=892796 TSecr=3119287179 [TCP PDU reassembled in 52]
524	94.907173885	192.168.5.128	192.168.5.130	TCP	66	40158 → 80 [ACK] Seq=727 Ack=2897 Win=70144 Len=0 TSval=3119287198 TSecr=892796
525	94.908005274	192.168.5.130	192.168.5.128	TCP	1514	80 → 40158 [ACK] Seq=2897 Ack=727 Win=7296 Len=1448 TSval=892796 TSecr=3119287179 [TCP PDU reassembled in 52]
526	94.908032593	192.168.5.128	192.168.5.130	TCP	66	40158 → 80 [ACK] Seq=727 Ack=4345 Win=72960 Len=0 TSval=3119287199 TSecr=892796
527	94.908444953	192.168.5.130	192.168.5.128	HTTP	5123	HTTP/1.1 200 OK (text/html)
528	94.908460792	192.168.5.128	192.168.5.130	TCP	66	40158 → 80 [ACK] Seq=727 Ack=9402 Win=80640 Len=0 TSval=3119287199 TSecr=892796
529	94.976417171	192.168.5.128	192.168.5.130	HTTP	597	GET /mutillidae/images/favicon.ico HTTP/1.1
530	94.976646182	192.168.5.128	192.168.5.130	TCP	66	40158 → 80 [FIN, ACK] Seq=1258 Ack=9402 Win=80640 Len=0 TSval=3119287267 TSecr=892796
531	94.977934824	192.168.5.130	192.168.5.128	TCP	1514	80 → 40158 [ACK] Seq=9402 Ack=1259 Win=8764 Len=1448 TSval=892813 TSecr=3119287267 [TCP PDU reassembled in 53]
532	94.977969745	192.168.5.128	192.168.5.130	TCP	54	40158 → 80 [RST] Seq=1259 Win=0 Len=0
533	94.978055824	192.168.5.130	192.168.5.128	HTTP	249	HTTP/1.1 200 OK (image/x-icon)
534	94.978061319	192.168.5.128	192.168.5.130	TCP	54	40158 → 80 [RST] Seq=1259 Win=0 Len=0
537	107.195584280	192.168.5.128	142.251.152.119	TLSv1.3	93	Application Data
538	107.196000190	142.251.152.119	192.168.5.128	TCP	60	443 → 51164 [ACK] Seq=6434 Ack=19512 Win=64240 Len=0
539	107.237855196	142.251.152.119	192.168.5.128	TLSv1.3	93	Application Data
540	107.272684431	192.168.5.128	142.251.152.119	TCP	54	51164 → 443 [ACK] Seq=19512 Ack=6473 Win=61320 Len=0
543	120.283361576	192.168.5.128	142.251.152.119	TLSv1.3	93	Application Data
544	120.284301148	142.251.152.119	192.168.5.128	TCP	60	443 → 34900 [ACK] Seq=5402 Ack=40000 Win=64240 Len=0
545	120.272669851	142.251.152.119	192.168.5.128	TLSv1.3	93	Application Data
546	120.312491804	192.168.5.128	142.251.152.119	TCP	54	34900 → 443 [ACK] Seq=40000 Ack=5441 Win=65535 Len=0
547	130.214348296	192.168.5.128	34.107.243.93	TLSv1.3	93	Application Data
548	130.214758210	34.107.243.93	192.168.5.128	TCP	60	443 → 43254 [ACK] Seq=862 Ack=2752 Win=64240 Len=0

Frame 517: Packet, 74 bytes on wire (592 bits), 74 bytes captured (592 bits) on interface e...
 Section number: 1
 Interface id: 0 (eth0)
 Encapsulation type: Ethernet (1)
 Arrival Time: May 23, 2026 10:51:25.321127756 EDT
 UTC Arrival Time: May 23, 2026 14:51:25.321127756 UTC
 Epoch Arrival Time: 1779547885.321127756
 [Time shift for this packet: 0.000000000 seconds]
 [Time delta from previous captured frame: 587.529 microseconds]
 [Time delta from previous displayed frame: 587.529 microseconds]

Transmission Control Protocol: Protocol
 Packets: 16353 - Displayed: 2928 (17.9%)
 Profile: Default

SYN and ACK flags indicated the establishment and acknowledgment of a TCP connection between the client (192.168.5.128) and the external/public server.

- The SYN (Synchronize) flag was used by the client to initiate the TCP

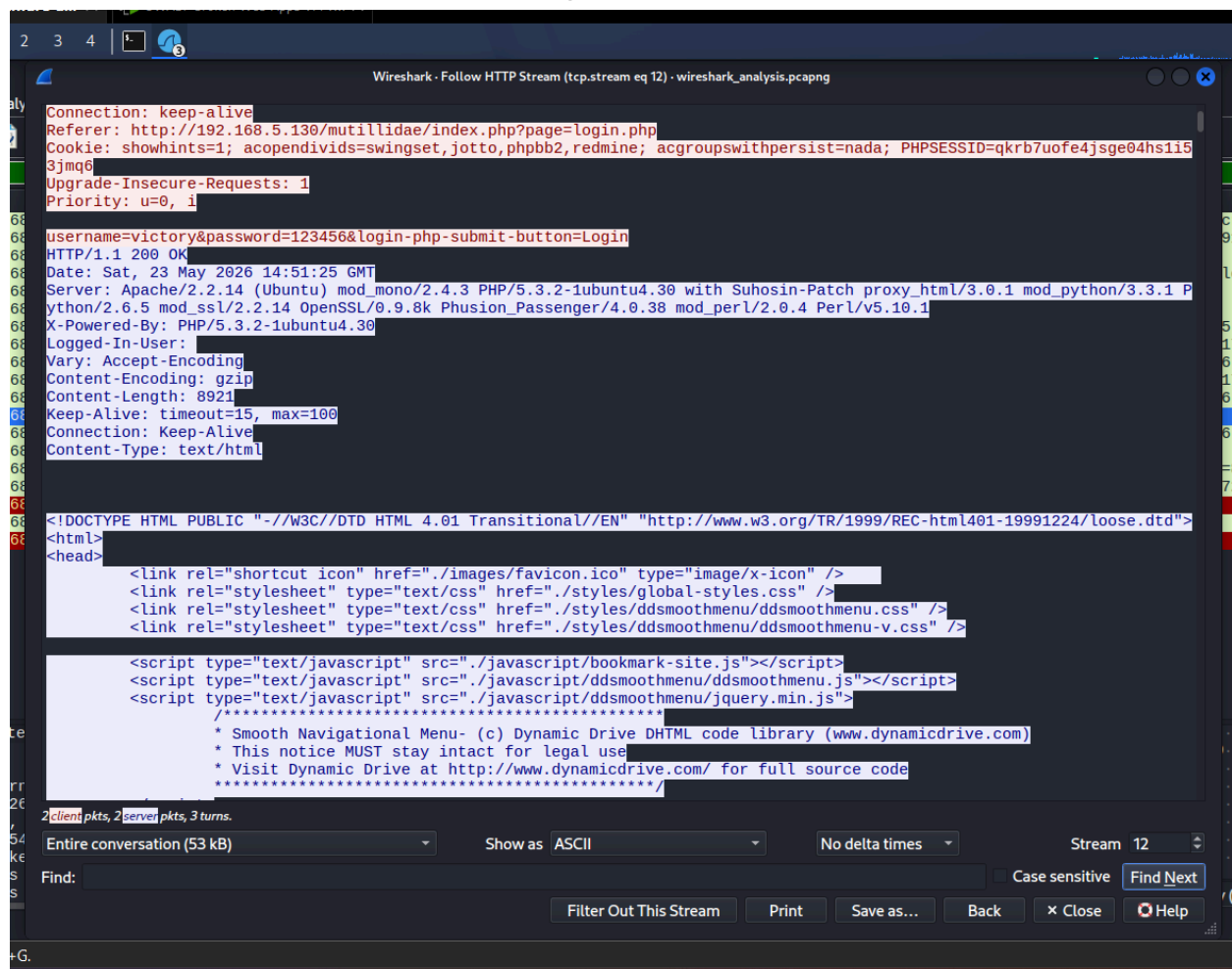
connection with the server.

- The ACK (Acknowledgment) flag confirmed the receipt of packets between both communicating hosts.

Exercise 2:

- Choose an HTTP packet and summarize its request method, status code, and any notable headers. What can you infer about the transaction?

Screenshot of HTTP packet analyzed.



The request method used was POST /mutillidae/index.php?page=login.php
HTTP/1.1

The server responded with the status code: HTTP/1.1 200 OK

Several notable headers were observed, including:

- Content-Type: application/x-www-form-urlencoded
- Cookie: PHPSESSID=...
- User-Agent: Mozilla Firefox
- Server: Apache/2.2.14 (Ubuntu)
- Connection: Keep-Alive

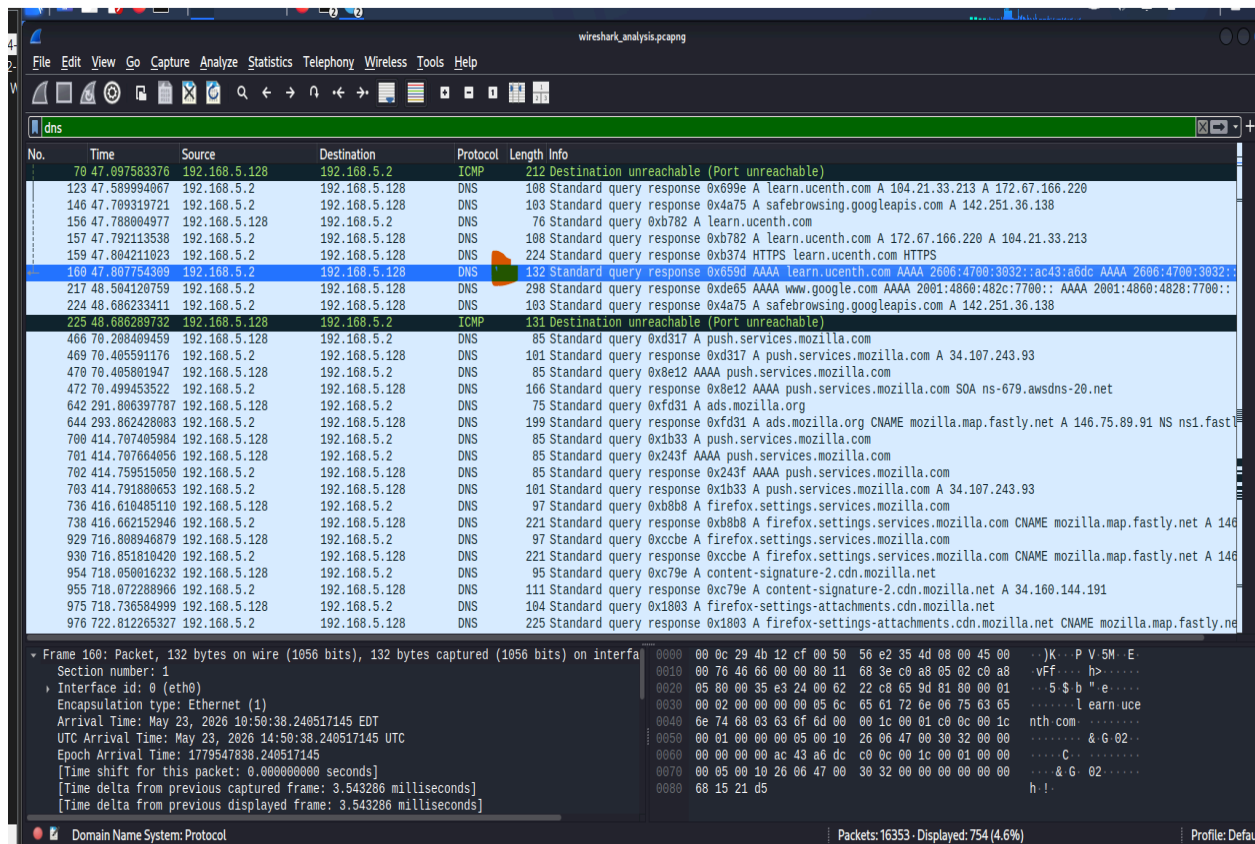
The packet also contained login credentials transmitted in plain text:
username=victory&password=123456

From this transaction, it can be inferred that the user attempted to authenticate to the Mutillidae web application using HTTP rather than HTTPS. This means sensitive information, including usernames, passwords, and session cookies, was exposed in clear text and could be intercepted by attackers monitoring the network traffic.

Exercise 3:

- Identify a DNS query and its corresponding response. What information does the response provide, and how is it structured?

Screenshot of Identified DNS query and response.



The DNS packet shown is a DNS response from 192.168.5.2 to 192.168.5.128.

The response provides:

- the requested domain name (learn.uceenth.com),
- the query type (AAAA record),
- and the resolved IPv6 addresses:

2606:4700:3032::ac43:a6dc

2606:4700:3032::6815:21d5

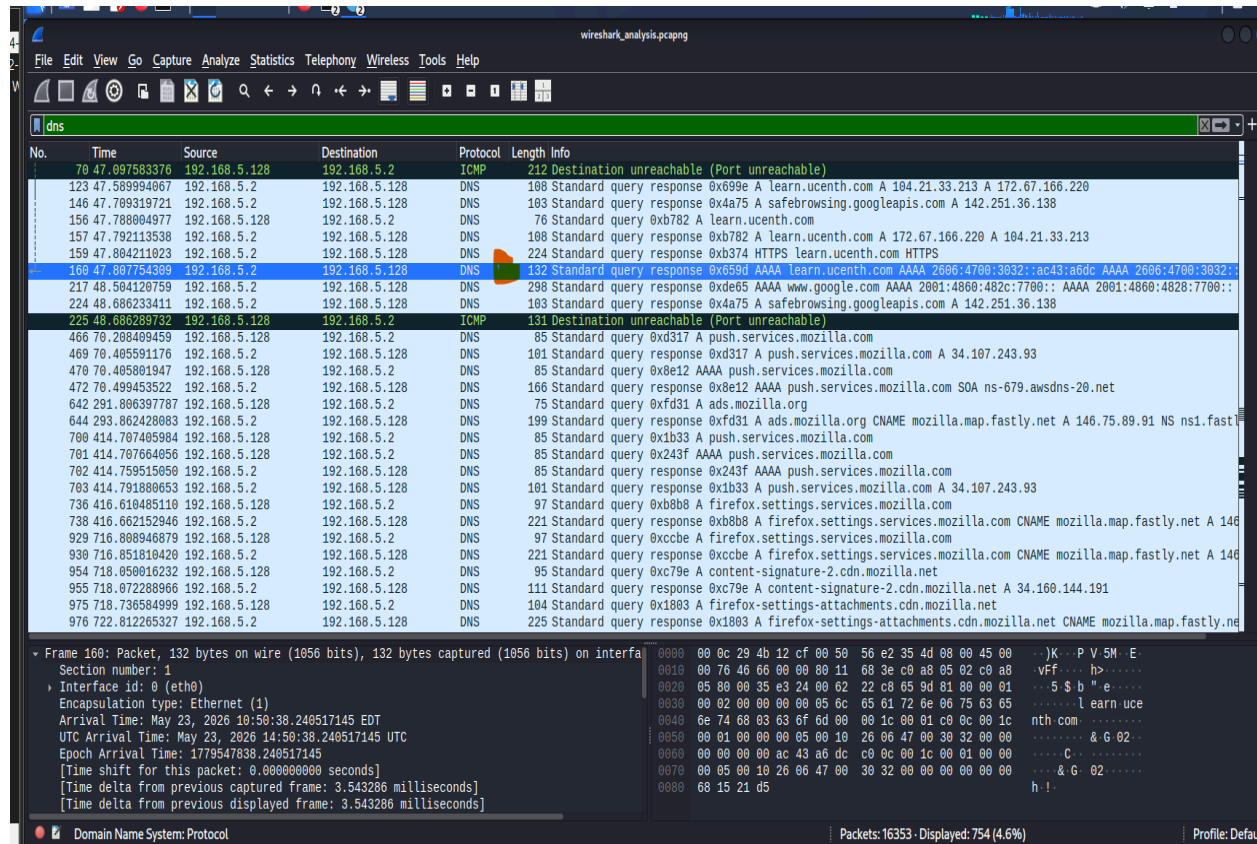
This means the client requested the IPv6 address of the domain, and the DNS server responded successfully with two valid IPv6 addresses associated with that hostname.

Exercise 4:

- Custom filter that captures only TCP traffic from my machine to a

specific target IP. Document the filter syntax and the packets captured.

Screenshot of custom filter that captures only TCP traffic

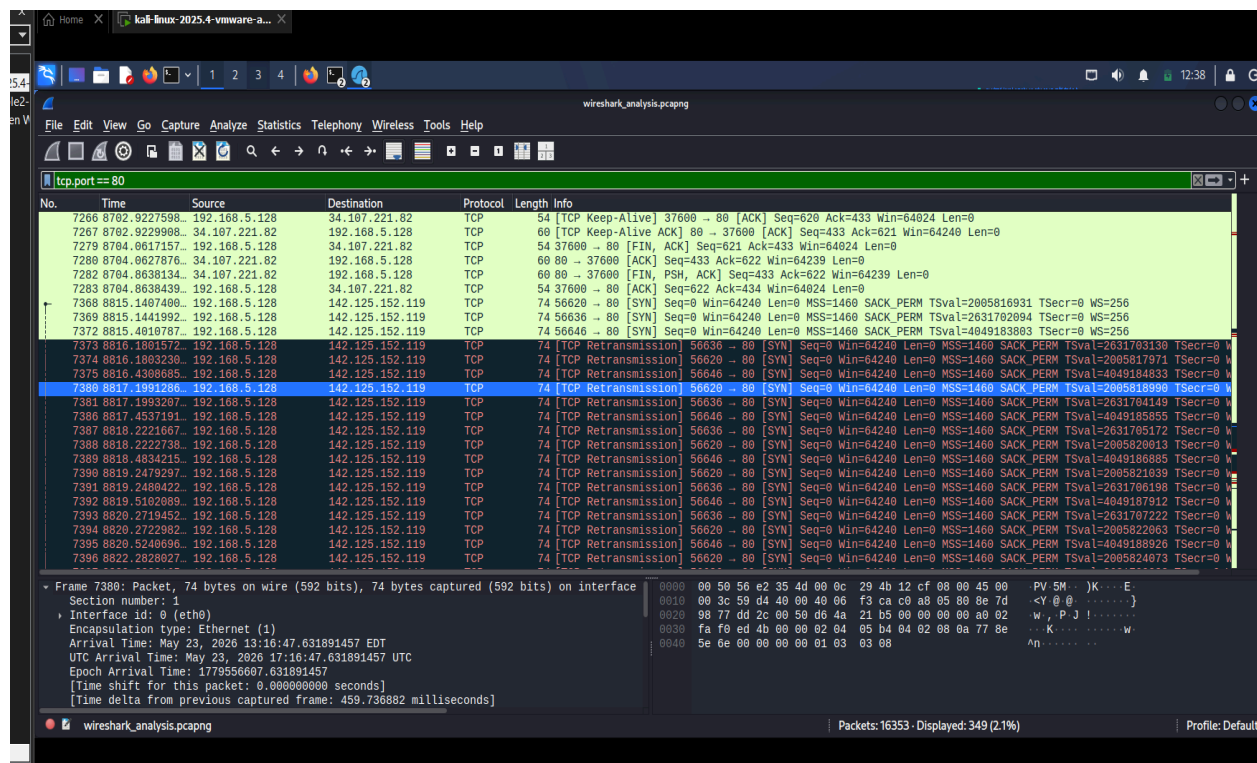


Custom Filter Syntax

tcp && ip.src == 192.168.5.128 && ip.dst == 142.125.152.119 The packets captured in this filter are primarily TCP SYN packets and TCP Retransmission SYN packets sent from client machine (192.168.5.128) to the target IP 142.125.152.119 on port 80 (HTTP).

Exercise 5:

- filter that captures traffic on a specific port (e.g., HTTP port 80) and analyzes the results. What packets were captured?



Filter Syntax

tcp.port == 80

The packets captured in this filter are TCP[Keep - Alive,ACK], TCP[FIN,PSH,ACK], TCP SYN packets and TCP Retransmission SYN packets sent from client machine (192.168.5.128) to the target IP 142.125.152.119 on port 80 (HTTP).

Exercise 6:

- Analyze your capture for any anomalies or indicators of potential vulnerabilities. Document your findings and suggest possible remediation steps.

Findings & Traffic Behavior Analysis

1. SYN and SYN Retransmissions

What this indicates:

- The client is attempting to establish a TCP connection (3-way

handshake).

- Retransmitted SYN packets indicated that the server did not respond to the initial SYN.

Possible causes:

- Packet loss on the network
- Firewall or security device silently dropping packets
- Target port is filtered or intermittently unreachable
- Server overload or rate-limiting SYN requests

Security implication:

- This can resemble a port scan or probing behavior
- Could indicate a filtered or protected HTTP service

2. TCP Keep-Alive ACK

What this means:

- A previously established connection is being maintained.
- The system is checking whether the remote host is still responsive.

Implications:

- Suggests at least some connections were successfully established earlier.
- Could also indicate idle connection monitoring by client or server.

3. TCP FIN, PSH, ACK

What this indicates:

- A connection is being closed properly (FIN handshake).
- PSH means data was actively pushed before closure.

Implication:

- At least one session successfully completed data exchange.
- Normal behavior for HTTP session termination.

Anomalies Detected

1. SYN Retransmissions (Key anomaly)

- Repeated SYN attempts without immediate response
- Suggests instability or filtering

This is the strongest indicator of a possible issue.

2. Mixed connection states:

- Failed connection attempts (SYN retries)
- Successful session behavior (FIN/PSH/ACK)
- Keep-alive traffic

This inconsistency may suggest:

- Intermittent network reachability
- Firewall rules blocking some sessions
- Load balancer or proxy behavior
- Unencrypted HTTP traffic (port 80)
- All traffic is over HTTP, not HTTPS

Risk:

- Data can be intercepted or modified (MITM attacks)

Recommended Remediation Steps

1. Upgrade to HTTPS (Critical)

- Migrate service from HTTP (port 80) to HTTPS (port 443)
- Use TLS certificates (Let's Encrypt or enterprise CA)

2. Investigate SYN retransmissions such as:

- Firewall rules
- IDS/IPS logs
- Server availability and load

3. Enable SYN protection

On server/firewall:

- Enable SYN cookies
- Configure SYN flood protection

4. Apply rate limiting

- Prevent excessive connection attempts from same IP

5. Harden HTTP service (if still required)

- Disable unused HTTP methods (e.g., PUT, DELETE)
- Restrict access by IP if internal-only service
- Enable logging and monitoring

6. Monitor with IDS/IPS

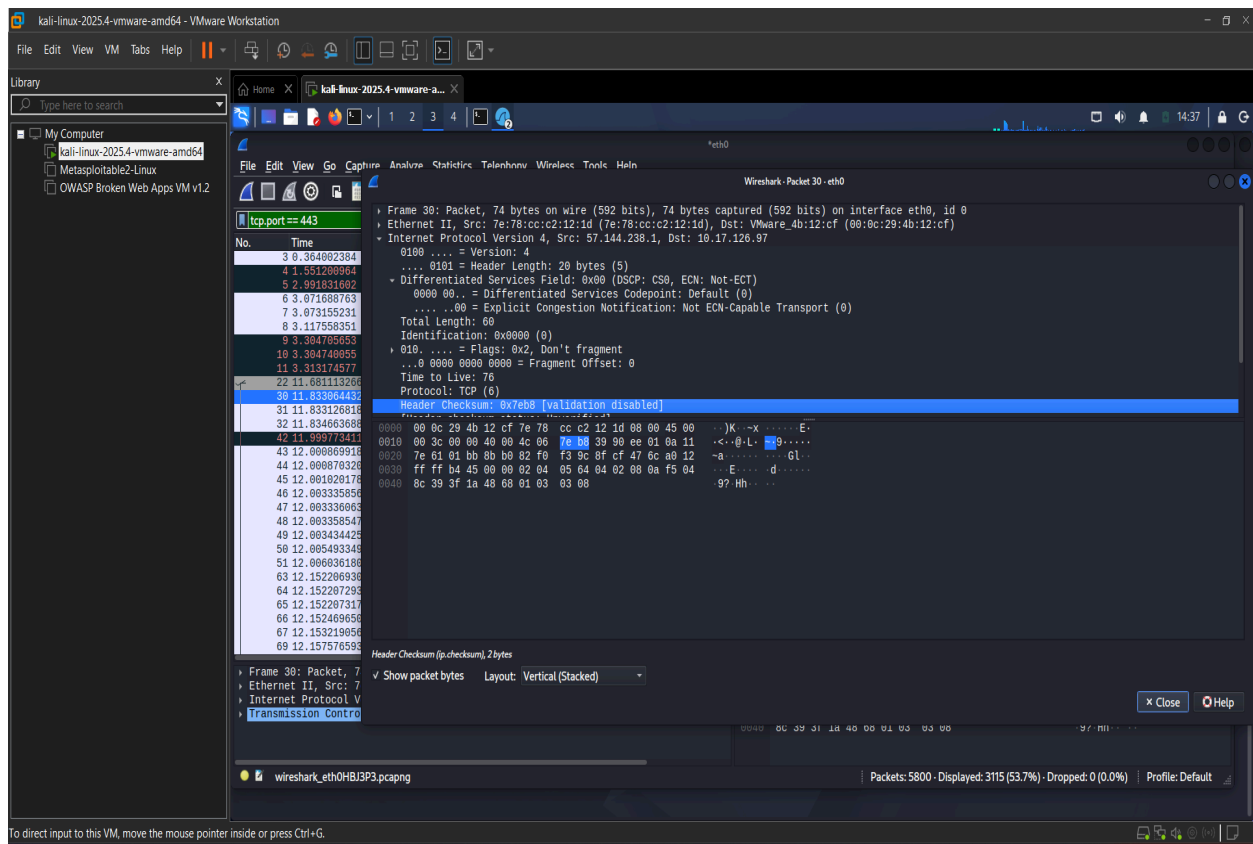
- Use tools like:

- Snort
- Suricata

- Alert on:
 - a. SYN retransmission spikes
 - b. repeated failed handshakes

Exercise 7:

- Capture HTTPS traffic and identify the initial handshake packets. What information is exchanged during this handshake, and how does it contribute to security?



What information is exchanged during handshake

During HTTPS communication, Wireshark captures the TLS handshake which includes packets such as Client Hello, Server Hello, Certificate, Key Exchange, and Finished messages. In this process, the client and server exchange supported TLS versions, cipher suites, random values, and the server's digital certificate. The certificate contains the server's public key and verifies its identity through a trusted Certificate Authority. After this

exchange, both parties derive a shared secret key using secure key exchange algorithms such as Diffie-Hellman.

How does it contribute to security

This handshake establishes an encrypted session, ensuring confidentiality, integrity, and authentication.

It protects against eavesdropping, data tampering, and man-in-the-middle attacks by ensuring that only the intended server can decrypt the communication and that transmitted data cannot be modified.

FINDINGS

- TCP 3-way handshake observed:

SYN → SYN-ACK → ACK

This confirms successful session establishment between systems.

- TCP flags identified:

- SYN → connection initiation
- SYN-ACK → server response
- ACK → acknowledgment of communication

- Open ports discovered:

- Port 22 → SSH (open)
- Port 80 → HTTP (open)
- Port 139 → NetBIOS/SMB (open)

- HTTP traffic:

- HTTP packets were captured successfully
- Traffic is unencrypted and readable
- This creates a risk of data exposure during transmission

- HTTPS traffic:

- HTTPS traffic could not be read due to encryption
- Only TLS handshake was visible
- Payload content was hidden

- Packet filtering:

- Applying filters (e.g., HTTP) reduced visible packets
- Removing filters restored full traffic

SECURITY ANALYSIS

Observed Weaknesses:

- Open SSH (22) may allow brute-force attacks
- HTTP traffic is unencrypted (MITM risk)
- SMB (139) can expose file sharing vulnerabilities
- Multiple exposed services increase attack surface
- Encrypted traffic reduces visibility for monitoring tools

RECOMMENDATIONS

- Replace HTTP with HTTPS (TLS encryption)
- Disable unused services (especially SMB/NetBIOS)
- Secure SSH with key-based authentication
- Apply firewall rules to restrict open ports
- Use IDS/IPS (e.g., Snort or Suricata)
- Regular monitoring and log analysis
- Perform periodic vulnerability assessments

Conclusion

The network analysis revealed multiple exposed services and insecure communication protocols within the Metasploitable environment. While the system is intentionally vulnerable for testing purposes, the findings demonstrate real-world risks such as open ports, unencrypted traffic, and potential attack vectors. Proper security controls such as encryption, Service hardening, and monitoring are essential to mitigate these risks.